

Finding the holes: Inspector, runtime monitoring and Amazon Q Developer

Four moments where a vulnerability can be caught – and one service the exam guide names that no longer exists.

A vulnerability can be caught at four moments. Each is cheaper than the next.

Four moments, four services

MOMENT	WHAT CATCHES IT
In the IDE, before commit	Amazon Q Developer security scans in the plugin
In the repository, at pull request	Amazon Q Developer code review; Inspector Code Security
In the artefact – image, package, function	Amazon Inspector enhanced scanning
At runtime, while it is being exploited	GuardDuty Runtime Monitoring

ON THE EXAM

Inspector finds the known CVE. GuardDuty sees the exploitation. Q Developer finds the flaw you were about to write.

The CodeGuru situation, stated plainly

- ◆ **Amazon CodeGuru Security was discontinued on 20 November 2025.** Console, resources and documentation are gone.
- ◆ **CodeGuru Reviewer stopped new repository associations on 7 November 2025.**
- ◆ The **SCS-C03 guide**, published 26 March 2026, still names **CodeGuru Security** in skill 3.2.6.
- ◆ AWS directs you to **Amazon Q Developer** for code review and **Amazon Inspector** for code scanning.
- ◆ **On the exam** it is probably the intended answer. **In practice it does not exist.**

ON THE EXAM

Exam guides lag reality. Knowing both positions is the honest one.

Amazon Q Developer for code review

- ◆ **Static analysis (SAST)** for vulnerabilities and best practice, in the IDE and on pull requests.
- ◆ **Secrets exposure detection** – the hardcoded credential, caught before it reaches git history.
- ◆ **Dependency vulnerability detection (SCA)**, plus code quality findings.
- ◆ Runs in the **IDE plugin** and in **GitHub and GitLab Duo** repositories.
- ◆ It reviews code. It knows nothing about deployed infrastructure.

ON THE EXAM

A secret caught in the IDE is deleted. One caught after commit lives in git history.

Amazon Inspector, extended

- ◆ Module 4 covered EC2, ECR images and Lambda. **Inspector Code Security** adds source repositories.
- ◆ It **discovers repositories in GitHub and GitLab automatically** and scans them for software vulnerabilities and unintended network exposure.
- ◆ So the same service now covers the repository, the built image and the running instance, with findings in one place.
- ◆ **SBOM export** gives you a software bill of materials for every monitored resource – increasingly a procurement requirement.
- ◆ **Inspector still never remediates.** The fix is a rebuild, a patch, or a developer.

ON THE EXAM

One finding pipeline from source to running workload.

GuardDuty Runtime Monitoring, in depth

- ◆ The **GuardDuty security agent** observes processes, files and network syscalls inside EC2, ECS including Fargate, and EKS.
- ◆ It sees what no scanner can: a **zero-day being exploited**, a reverse shell, a cryptominer.
- ◆ Deploy via **EKS add-ons**, ECS, or automated agent configuration. Bake it into the AMI.
- ◆ **Required** for ECS attack sequence findings; recommended with EKS Protection.
- ◆ Coverage is not automatic. An instance without the agent produces nothing, silently.

ON THE EXAM

Scanning says a vulnerability exists. Runtime says someone is using it.

Vulnerability, or exploitation?

SCANNING TELLS YOU

This could be exploited

- A known CVE in a package or image
- A hardcoded secret in source
- Found before anything happens
- Inspector, Q Developer

RUNTIME TELLS YOU

This is being exploited

- A process that should not exist
- Behaviour, not a signature
- Found while it is happening
- GuardDuty Runtime Monitoring

THE TIEBREAKER

A scanner reports a possibility. Runtime monitoring reports an event. Only the second one wakes anybody up.

Securing the pipeline itself

- ◆ A pipeline that can deploy anything is a credential that can do anything.
- ◆ **One scoped role per pipeline**, able to deploy that workload and nothing else.
- ◆ **Manual approval** before production; separate build and deploy accounts.
- ◆ **Sign artefacts** and verify at deploy, so what runs is what the pipeline produced.
- ◆ Do not let a build job read production secrets.

ON THE EXAM

Every scanner assumes the artefact reaching production is the one that was scanned.

A fintech's security team reads that AWS offers CodeGuru Security and asks its developers to adopt it for static analysis, secrets detection and dependency scanning in the IDE and on pull requests.

The developers report that they cannot find the service in the console and that the documentation links return errors. The team asks what they should actually use.

What should the team be told?

- A CodeGuru Security is only available in certain Regions; switch to us-east-1
- B CodeGuru Security was discontinued in November 2025; use Amazon Q Developer for code review
- C CodeGuru Security requires an Enterprise Support plan before it appears in the console
- D CodeGuru Security has been folded into GuardDuty Runtime Monitoring

◆ PAUSE AND DECIDE

What should the team be told?

- A

 Region availability; switch to us-east-1
A common shape of answer for a genuinely regional service. The console and documentation are gone everywhere, which no Region setting explains.
- B

 Discontinued November 2025; use Amazon Q Developer
Correct. AWS ended support on 20 November 2025 and directs customers to Q Developer for code review and Inspector for code scanning.
- C

 It needs Enterprise Support
Invented. Support plans gate the Shield Response Team, as module 8 covered - they do not gate whether a service appears in the console.
- D

 Folded into GuardDuty Runtime Monitoring
Different moment entirely. Runtime Monitoring observes processes inside running workloads; it has never analysed source code.

THE SIGNAL

'Documentation links return errors' is the tell. A Region or permissions problem never removes the documentation.

A software company keeps 340 repositories in GitHub. It wants continuous discovery and scanning of those repositories for vulnerable dependencies and for code that unintentionally exposes resources to the network, with findings arriving alongside its existing EC2 and container findings.

The security team already uses Amazon Inspector for EC2, ECR and Lambda, and reviews findings in Security Hub.

What should the company enable?

- ☐ A Amazon Inspector Code Security, which discovers and scans GitHub and GitLab repositories
- ☐ B GuardDuty Runtime Monitoring on the build agents that clone the repositories
- ☐ C A Config rule evaluating repository configuration against a conformance pack
- ☐ D Macie automated sensitive data discovery pointed at the repository mirrors in S3

◆ PAUSE AND DECIDE

What should the company enable?

- A

Inspector Code Security
Correct. It discovers GitHub and GitLab repositories automatically and scans for software vulnerabilities and unintended network exposure, into the Inspector findings they already use.
- B

Runtime Monitoring on build agents
Watches processes on the build agent. It would notice something malicious running during a build, and it never inspects the repository contents.
- C

Config rule with a conformance pack
AWS Config evaluates AWS resource configuration. A GitHub repository is not an AWS resource and is not in the configuration recorder.
- D

Macie on repository mirrors in S3
Macie finds sensitive data in S3 objects. It answers 'is there personal data here', not 'is this dependency vulnerable'.

THE SIGNAL

'Alongside its existing findings' is the constraint that picks Inspector over an equally capable third-party scanner.

A retailer scans every container image with Inspector enhanced scanning and blocks any build with a HIGH or CRITICAL finding. All images in production are currently clean.

An attacker exploits a previously unknown flaw in the application's request parsing, gains code execution inside a container, and runs a cryptominer. No scan had anything to report, because no CVE existed for the flaw.

What would have detected this?

- ☐ A Increasing Inspector scan frequency on the ECR repository
- ☐ B Amazon Q Developer static analysis on the application source
- ☐ C An Inspector SBOM export reviewed weekly by the security team
- ☐ D GuardDuty Runtime Monitoring on the ECS tasks

◆ PAUSE AND DECIDE

What would have detected this?

A Scan the ECR repository more often

Scanning compares against known CVEs. Doing that more frequently against a vulnerability that has no CVE returns the same clean result, faster.

C Weekly SBOM review

An SBOM lists what is in the image. Every component was legitimate and none was known-vulnerable, so the SBOM would look perfect.

B Q Developer static analysis

The best of the wrong answers – it might have flagged the parsing flaw before release. The question asks what would have detected the attack, and static analysis runs before deployment.

D GuardDuty Runtime Monitoring on the ECS tasks

Correct. It observes behaviour rather than signatures, so a cryptominer process that was never in the image is exactly what it detects.

THE SIGNAL

'No CVE existed' eliminates every signature-based control at once. What remains is behaviour.

A media company runs a single CI/CD pipeline role used by all 40 of its services. The role holds permissions to deploy to every account, read every secret and update every Lambda function, because that was simplest when the platform was built.

A security review notes that all code and images are thoroughly scanned before deployment, and asks what residual risk remains.

What is the MOST significant residual risk?

- ☐ A Scanning may miss a zero-day vulnerability in a dependency
- ☐ B Inspector findings may not reach Security Hub in all Regions
- ☐ C Compromising the pipeline role lets an attacker deploy unscanned code to any account
- ☐ D SBOM exports are not retained long enough for an audit

◆ PAUSE AND DECIDE

What is the MOST significant residual risk?

- A

 Scanning may miss a zero-day
True of all scanning and it is the risk they have partly mitigated with runtime monitoring. It also does not depend on anything specific to this scenario.
- B

 Findings may not reach Security Hub everywhere
A real operational gap worth checking, and a reporting problem rather than the largest risk in a design with one omnipotent role.
- C

 Compromising the pipeline role deploys unscanned code anywhere
Correct. The scanning is a gate on one path. A role that can deploy to every account is a second path that bypasses every gate.
- D

 SBOM retention
A compliance detail. It affects what you can prove afterwards, not what an attacker can do.

THE SIGNAL

Every scanner assumes the artefact reaching production came through the pipeline. One over-privileged role invalidates that assumption.

Five things to carry forward

- ◆ **Four moments: IDE, repository, artefact, runtime.** The scenario always tells you which one.
- ◆ **CodeGuru Security was discontinued on 20 November 2025** – and the exam guide still names it.
- ◆ **Q Developer reviews code. Inspector scans artefacts and repositories. GuardDuty watches runtime.**
- ◆ **'No CVE existed' eliminates every scanner.** What is left is behaviour.
- ◆ **An over-privileged pipeline role bypasses every scan you built.** Scoped role per pipeline.

ON THE EXAM

Next: securing generative AI workloads – Bedrock guardrails and the OWASP Top 10 for LLM Applications.